# Social Media Analytics Engine : Review 2 material

---

## 1. DDL with constraints

The full script is `db/schema.sql`. Every table, and what each constraint is doing:

| Table | Constraints |
|---|---|
| `users` | `user_id` PK (AUTO_INCREMENT); `username` UNIQUE; `email` UNIQUE; `CHECK (email LIKE '%@%.%')`; all columns except `bio` NOT NULL |
| `posts` | `post_id` PK; `user_id` FK `users` ON DELETE CASCADE; `CHECK (post_type IN ('text','image','video','link'))` |
| `comments` | `comment_id` PK; `post_id` FK `posts` CASCADE; `user_id` FK `users` CASCADE |
| `followers` | Composite PK `(follower_id, followee_id)`; both columns FK `users` CASCADE; `CHECK (follower_id <> followee_id)` |
| `interactions` | `interaction_id` PK (BIGINT); `post_id` and `user_id` FKs CASCADE; `CHECK (interaction_type IN ('like','share','save','view'))`; `UNIQUE (post_id, user_id, interaction_type)` |

Row-level and column-level constraints present: PRIMARY KEY, FOREIGN KEY with referential action, UNIQUE (single and composite), CHECK (value list and pattern), NOT NULL, AUTO_INCREMENT, DEFAULT.

Supporting tables created by `db/programs.sql`: `interaction_audit`, `post_audit`, `user_engagement_report`, `trending_report`.

Three views: `v_post_engagement`, `v_user_engagement`, `v_daily_engagement`.

---

## 2. Query requirements, stated in English

Each one is implemented in `queries/catalog.js` and runnable from the Query lab page.

| # | Requirement | SQL techniques | Catalog id |
|---|---|---|---|
| 1 | Find the top-performing posts by weighted engagement score. | JOIN, GROUP BY, HAVING, conditional aggregate | `top_posts` |
| 2 | Rank creators by total engagement, overall and within their country. | CTE, RANK, DENSE_RANK, PERCENT_RANK, PARTITION BY | `rank_users` |
| 3 | Identify trending posts: the ones collecting activity in the last seven days, whenever they were published. | CTE, UNION ALL, weighted SUM, DENSE_RANK, date filtering | `trending` |
| 4 | Compare each post against the average of its own author's posts. | Window AVG with PARTITION BY, CASE, CTE | `post_vs_author_avg` |
| 5 | Calculate daily engagement totals with a 7-day moving average. | Window frame ROWS BETWEEN, LAG, running total | `daily_trend` |
| 6 | Show the top three posts for every active creator. | ROW_NUMBER with PARTITION BY, filtering on a window result | `top_per_author` |
| 7 | Find users with more followers than the platform average. | Correlated subquery, scalar subquery, derived table | `above_avg_followers` |
| 8 | Analyse engagement by hour of day to find peak activity. | HOUR(), GROUP BY, SUM() OVER () for share of total | `hourly_pattern` |
| 9 | Identify accounts that have never interacted with anything. | NOT EXISTS anti-join | `quiet_users` |
| 10 | Break the busiest posts down by interaction type. | Window SUM OVER PARTITION, percentage | `interaction_mix` |

| | | | |
|---|---|---|---|
| | | of total | |
| 11 | List pairs of users who follow each other back. | Self join on a composite key, duplicate-pair filter | `mutual_follows` |
| 12 | Compare average engagement across post formats. | Aggregate functions, GROUP BY, ratio comparison | `type_performance` |
| 13 | Show follower growth month by month, cumulatively. | DATE_FORMAT, running total with SUM() OVER | `follower_growth` |
| 14 | Find who has been most active in the last three days. | Narrow date filter, JOIN, GROUP BY, conditional aggregate | `recent_activity` |

---

## 3. Stored functions

### `CalculatePostEngagement(post_id) RETURNS DECIMAL(10,1)`

**What it does.** Returns the weighted score for one post: view 0.2, like 1, save 2, comment 2, share 3.

**Why it exists.** The scoring rule appears in the dashboard, the leaderboard, both procedures and several queries. Putting it in one function means changing the weights in one place instead of nine. Verified against `v_post_engagement` — both return the same number.

**Where it is used.** Called by both procedures, by the `/api/functions/post/:id` endpoint, and usable in any query: `SELECT CalculatePostEngagement(2341);`

### `CalculateUserEngagement(user_id) RETURNS DECIMAL(12,1)`

**What it does.** Totals the points across everything a creator has published, by calling the first function for each of their posts.

**Why it exists.** Creator-level scoring is needed by the leaderboard and the report procedure. It also demonstrates one stored function calling another.

### `CalculateWindowEngagement(post_id, start, end) RETURNS DECIMAL(10,1)`

**What it does.** The same weighted score, but counting only the interactions and comments that happened *between two moments*.

**Why it exists.** Lifetime score and trending score are different questions. A post from March with 300 points earned in March is not trending today. `GenerateTrendingPostReport` uses this function so that the procedure and the Query lab's trending query mean exactly the same thing — verified: both return an identical top 20.

---

## 4. Stored procedures, both using cursors

### `GenerateUserEngagementReport(IN p_user_id INT)`

**Scenario.** A creator opens their analytics page and asks for a summary of their whole catalogue.

**How the cursor is used.** The procedure declares a cursor over every post belonging to that user, opens it, and fetches one `post_id` at a time inside a loop. For each row it calls `CalculatePostEngagement`, adds to a running total, counts the post, and keeps track of the best performer so far. A `CONTINUE HANDLER FOR NOT FOUND` ends the loop. The summary is written to `user_engagement_report` with `REPLACE INTO`, then selected back.

**Why a cursor is justified.** The procedure carries state between rows — running total, post count, best post so far — and calls a function per row. A cursor makes that row-by-row logic explicit.

**Error handling.** If the user id does not exist, the procedure raises `SIGNAL SQLSTATE '45000'` with the message "No user with that id".

### `GenerateTrendingPostReport(IN p_start DATETIME, IN p_end DATETIME, IN p_limit INT)`

**Scenario.** The editorial team wants the trending list for a chosen week or month.

**What "trending" means here.** Posts that *received activity* in the window, not posts *published* in the window. An older post people are reacting to this week is trending; a post published this week that nobody touched is not. The cursor therefore selects posts with an interaction or comment `BETWEEN p_start AND p_end`, and scores each one with `CalculateWindowEngagement` rather than the lifetime function.

**How the cursor is used.** The loop fetches each qualifying post, scores its window activity, and inserts it into `trending_report` only if it earned anything. After the cursor closes, the procedure returns the ranked list with `RANK() OVER (ORDER BY points DESC)`.

**Consistency check worth demonstrating.** The procedure and the `trending` query in the Query lab return an identical top 20 with identical scores — one computed row by row with a cursor, the other in a single set-based statement. Same definition, two implementations.

**Why a cursor is justified, and what it costs.** Each row is scored and conditionally written. It also shows the price of row-by-row processing honestly: the cursor version takes a few hundred milliseconds where the set-based query takes about 60 ms. That comparison is a good thing to raise before an examiner raises it.

---

## 5. Triggers

### `trg_no_self_engagement` — BEFORE INSERT ON `interactions`

**Purpose.** Integrity. A user must not like, save, share or view their own post.

**Why it has to be a trigger.** The rule compares the incoming `user_id` against `posts.user_id`, a value in another table. A CHECK constraint cannot read another table, so this is exactly the case a trigger exists for. On violation it raises `SIGNAL SQLSTATE '45000'` and the insert is rejected.

### `trg_interaction_audit` — AFTER INSERT ON `interactions`

**Purpose.** Auditing. Every engagement event is copied into `interaction_audit` with its timestamp, giving a tamper-evident record separate from the live table.

**Why it is not duplicated data.** It records *events*, not *counts*. Engagement totals are still derived from the base tables, so nothing can drift out of sync.

### `trg_post_delete_audit` — AFTER DELETE ON `posts`

**Purpose.** Auditing. Deleting a post cascades to its comments and interactions, so the rows themselves disappear. This trigger keeps a record of what was removed, by whom and when.

**Note on load order.** The application creates the triggers *after* the bulk data load, the way a real deployment does. There is no value in writing 177,000 audit rows for generated seed data.

---

## 6. Frontend, backend and connectivity

```
Browser dashboard      Node.js + Express API      MySQL 8.0
 (HTML, CSS, JS)       (mysql2 driver)            tables, views, indexes,
 no build step         parameterised queries      functions, procedures, triggers
```

- **Frontend:** the existing analytics dashboard — Overview, Explore, Query lab, SQL console, Index lab, Procedures & triggers, Schema. No data is hardcoded; every page fetches from the API.
- **Backend:** Express, using the `mysql2` connection pool. Query text lives server-side in `queries/catalog.js`; the browser sends an id, never SQL. Search endpoints use `?` placeholders.
- **Database:** MySQL. Connection settings come from `.env`.

**CRUD flow to demonstrate (Review 3):** on the Procedures & triggers page — create a post (INSERT), read it back in the lists (SELECT), react to it (INSERT into interactions, audit trigger fires, self-like refused by the integrity trigger), delete it (DELETE, cascade plus delete-audit trigger).

---

## 7. Indexing and optimisation

Nine indexes, each tied to a query pattern, defined in `db/indexes.js`.

**The MySQL-specific point worth making:** InnoDB creates an index for every foreign key automatically, so `posts.user_id`, `comments.post_id`, `comments.user_id`, `interactions.user_id` and `followers.followee_id` are already indexed by the constraints. PostgreSQL does not do this. The nine indexes here therefore cover what foreign keys do not: timestamps, post types, topics and countries.

Measured on 177,342 interactions, average of three runs:

| Query | Without project indexes | With them | Change | EXPLAIN access type |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| Who has been active in the last 3 days | 86 ms | 38 ms | 2.3× faster | `ALL` `range` |
| Trending in the last 7 days | 65 ms | 61 ms | no real change | scan either way |
| Daily activity, 45 days | 72 ms | 67 ms | no real change | scan either way |
| Accounts that never engage | 12 ms | 12 ms | already fast | foreign key's own index |

Evidence comes from `EXPLAIN`, shown in the Index lab: access type `ALL` is a full table scan, `range` and `ref` mean an index was used.

**Conclusion to state in the report:** indexes pay off when a query needs a small, selective slice of a large table. When a query reads most of the table, a full scan is cheaper and the optimiser ignores the index. Indexes also cost storage and slow down every INSERT, because each one must be maintained.

---